Chap 5

Trees

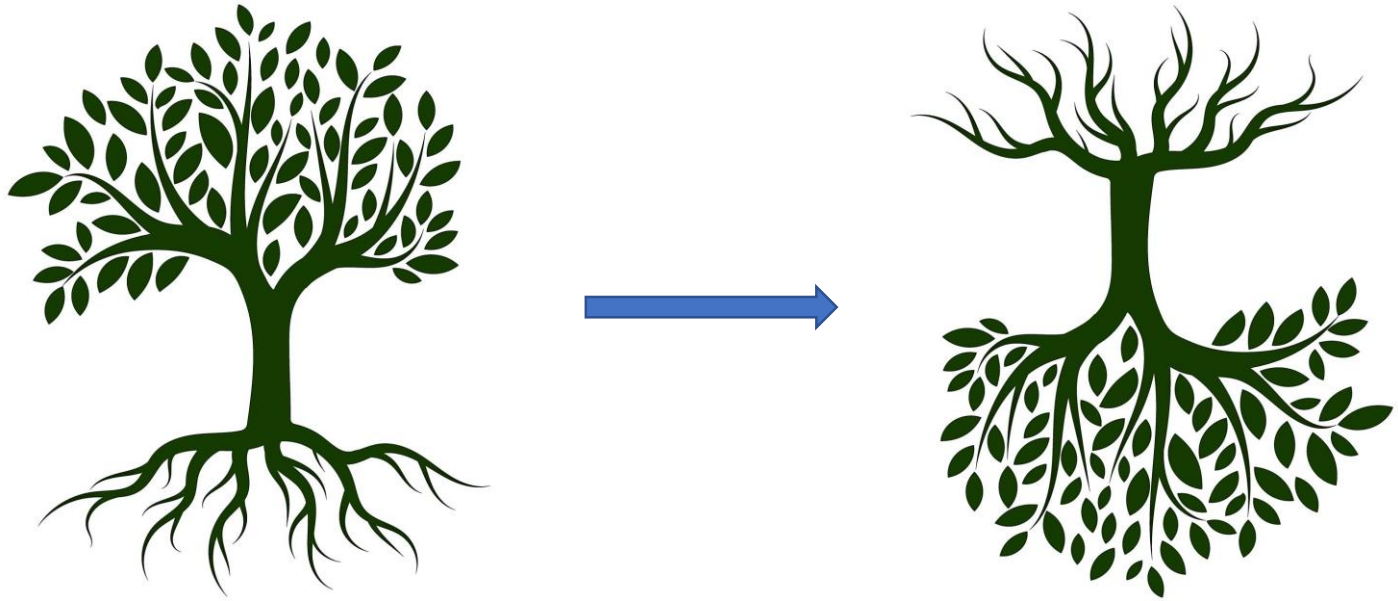
海大資訊工程系
蔡東佐

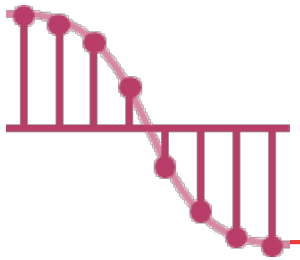
Outline

- Introduction（簡介）
- Binary Trees（二元樹）
- Binary Trees Traversals（二元樹之走訪）
- Additional Binary Tree Operations（更多的二元樹運算）
- Threaded Binary Trees（引線二元樹）

Introduction

- In this chapter we will study a very important data object, the **tree**.
- Intuitively, a tree structure means that the data are organized **in a hierarchical manner**.



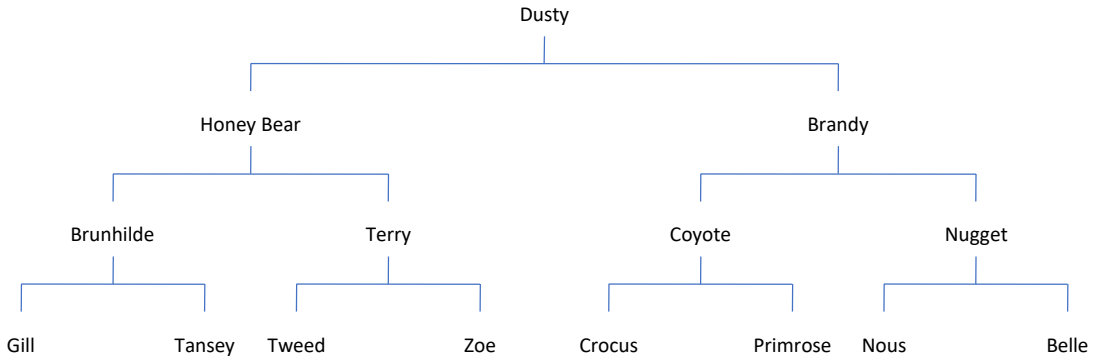


Tree - Two Examples 1/2

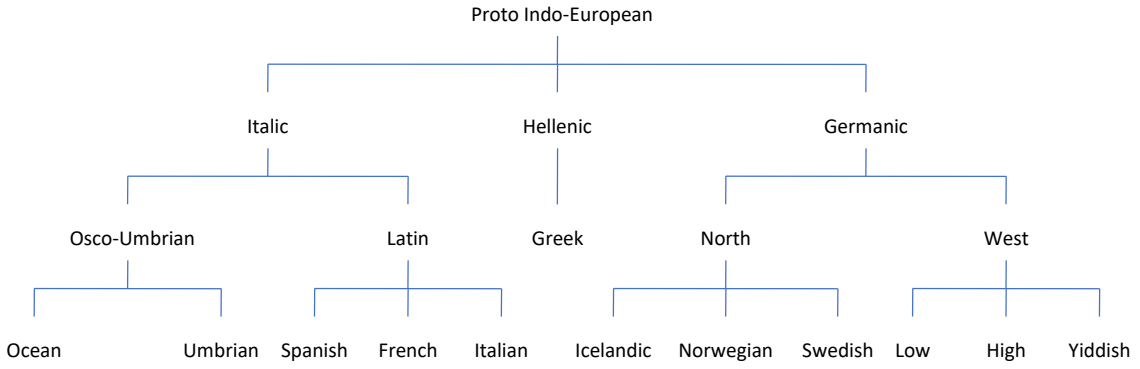
- Example 1. One very common place where such a structure arises is in the investigation of **genealogies**.

Tree - Two Examples 2/2

- Example 2. There are two types of genealogical charts that are used to present such data: the **pedigree** chart and the **lineal** chart.



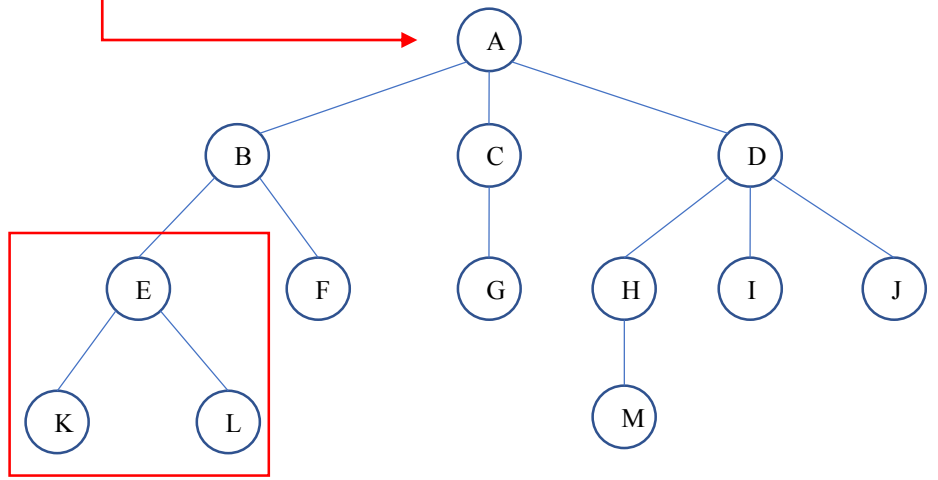
Dusty's parents are Honey Bear and Brandy.



The ancestry of the modern European languages.

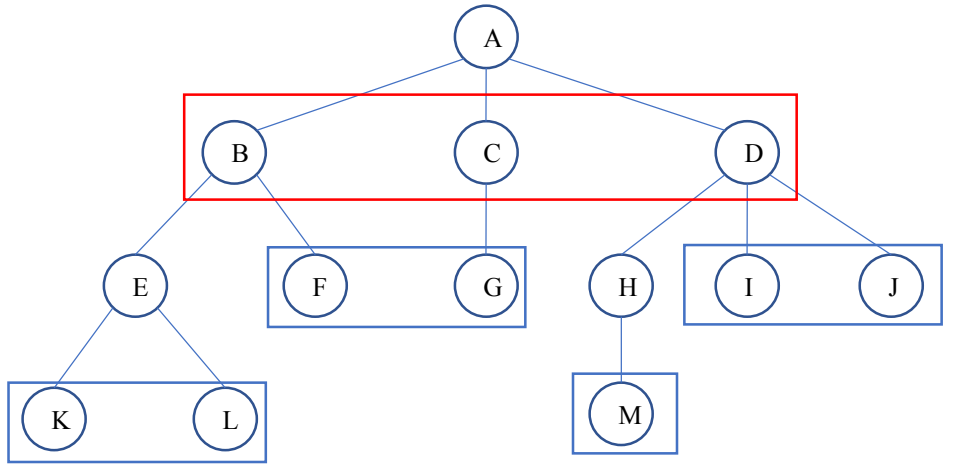
Definition 1/3

- A **tree** is a finite set of one or more nodes such that
  - ✓ There is a specially designated node called **root**. (根)
  - ✓ The remaining **nodes** are partitioned into $n \geq 0$ disjoint sets, $T_1,\ldots,T_n$, where each of these sets is a tree.
  - ✓ $T_1,\ldots,T_n$ are called **subtrees** (子樹) of the root.
- A **node** stands for the item of information plus the branches to other nodes.



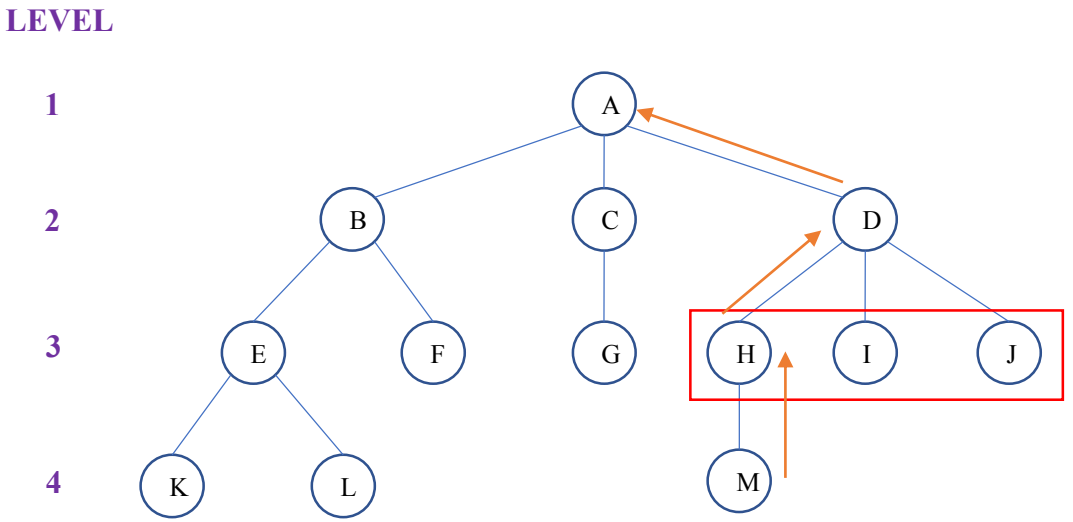
Definition 2/3

- The number of subtrees of a node is called its **degree**.
  - ✓ The degree of *A* is 3, of *C* is 1, and of *F* is zero.
  - ✓ The **degree of a tree** is the maximum of the degree of the nodes in the tree.
    - The tree has degree 3.
- Note that have degree zero are called **leaf** or **terminal nodes**. Consequently, the other nodes are referred to as **nonterminals**.
  - ✓ {K, L, F, G, M, I, J} is the set of leaf nodes.
- The roots of the subtrees of a node *X* are the **children** of *X*. *X* is the **parent** of its children.
  - ✓ The children of D are H, I, and J. The parent of D is A.



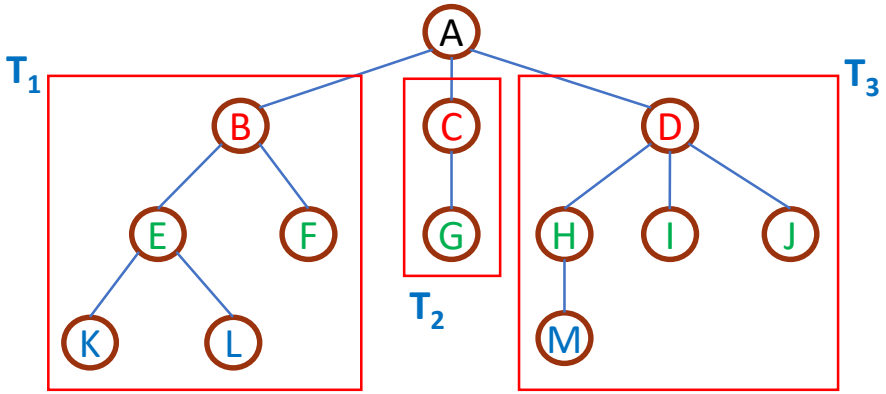
Definition 3/3

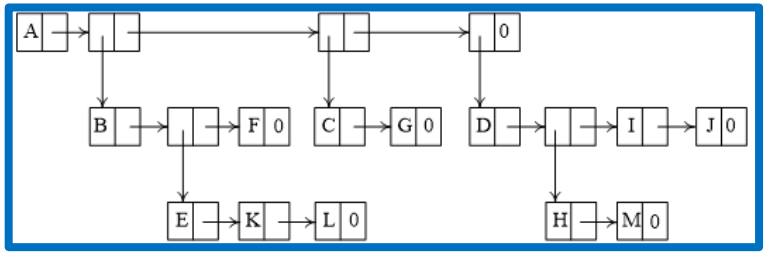
- Children of the same parent are said to be **siblings**. 兄弟
  - ✓ H, I, and J are siblings.
- The **ancestors** 祖先 of a node are all the nodes along the path from the root to that node.
  - ✓ The ancestors of M are A, D, and H.
- The **height** 樹高 or **depth** of a tree is defined to be the maximum level of any node in the tree.
  - ✓ The depth of the tree is 4.
  - ✓ The level of a node is defined by letting the root be at level one.
    - If a node is at level *k*, then its children are at level *k*+1.



Representation of Trees – List Representation

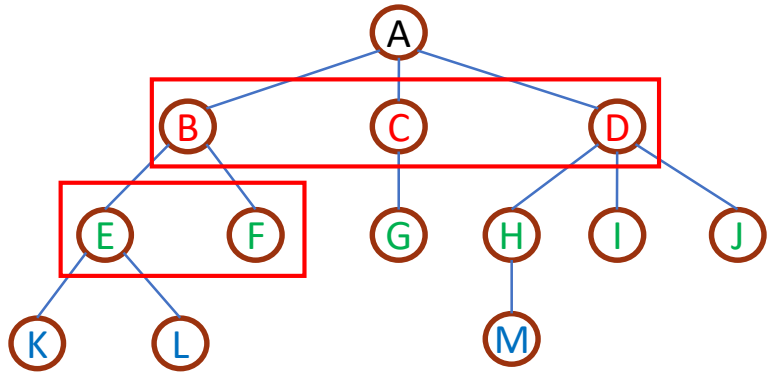
- The tree of Figures 5.2 could be written as the list
  - ✓(*A*(*B*(*E*(*K*, *L*), *F*), *C*(*G*), *D*(*H*(*M*), *I*, *J*)))
- The root node comes first, followed by a list of the subtrees.





Possible node structure of a tree of degree *k*

- Since the degree of each tree node may be different, we may be tempted to use memory nodes with a varying number of pointer fields.
- However, one only uses nodes of a fixed size to represent tree nodes in practice.



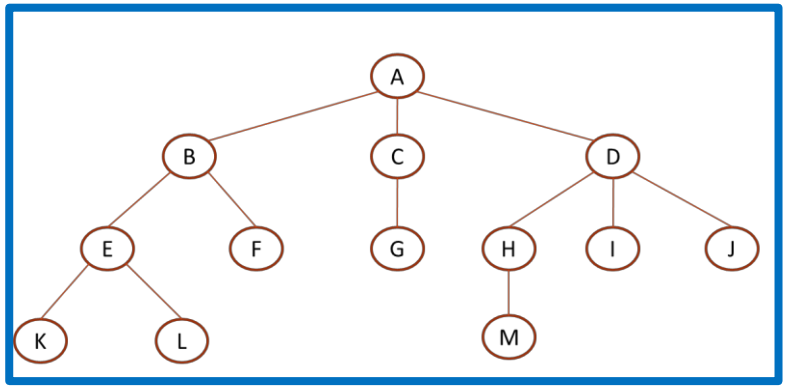
DATA	CHILD 1	CHILD 2	…	CHILD *k*

How to choose the fixed size?

Lemma 5.1 1/2

- If T is a k-ary tree (i.e., a tree of degree k) with n nodes ($n \geq 1$), each having a fixed size, then $n(k-1)+1$ of the nk child fields are 0.

DATA	CHILD 1	CHILD 2	…	CHILD k



- Example.
  - ✓Since the number of edges of T is exactly n-1=12, the number of non-zero child fields in T is exactly n-1=12 too.
  - ✓The total number of child fields in a 3-ary tree with 13 nodes is $nk=39$.
  - ✓Hence, the number of zero fields is $nk-(n-1) = n(k-1)+1=27$.

Lemma 5.1 2/2

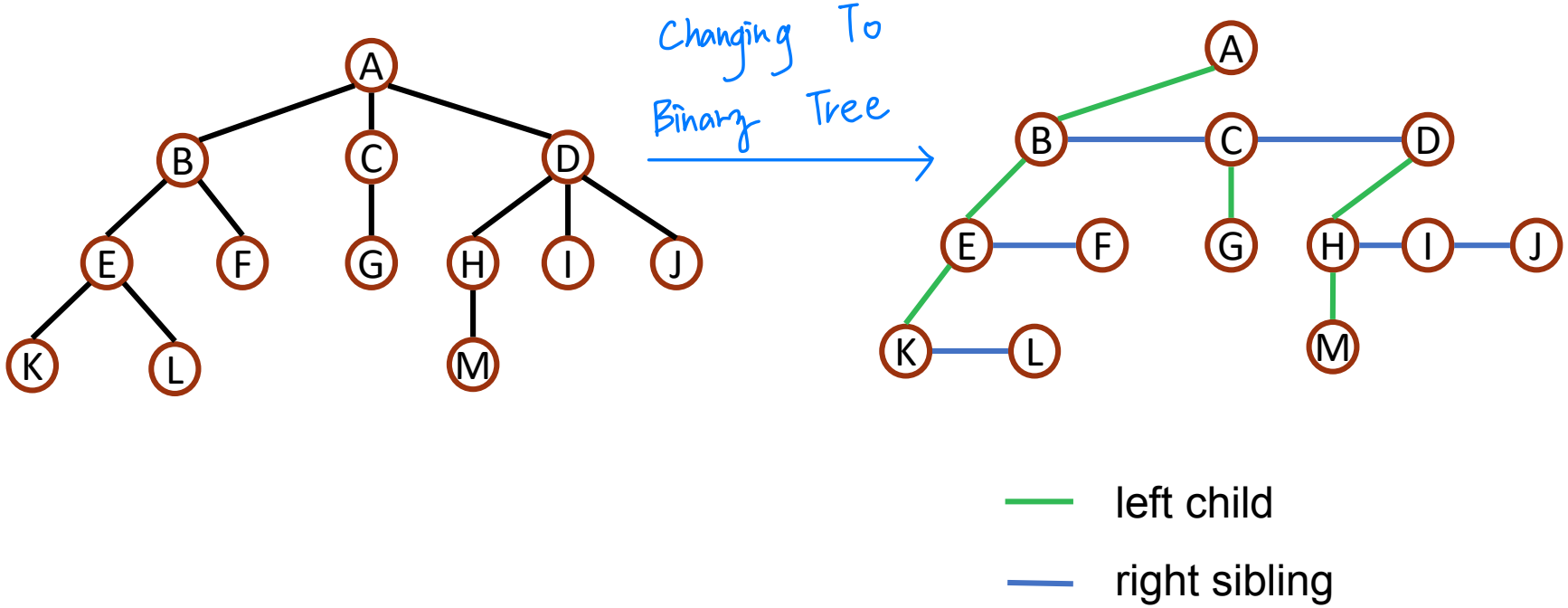
- If T is a k-ary tree (i.e., a tree of degree k) with n nodes ($n \geq 1$), each having a fixed size, then $n(k-1)+1$ of the nk child fields are 0.

DATA	CHILD 1	CHILD 2	…	CHILD k

- Proof.
  - ✓Since the number of edges of T is exactly $n-1$, the number of non-zero child fields in T is exactly $n-1$, too.
  - ✓The total number of child fields in a k-ary tree with n nodes is nk.
  - ✓Hence, the number of zero fields is $nk-(n-1) = n(k-1)+1$.

Representation of Trees – Left Child-Right Sibling Representation 1/2

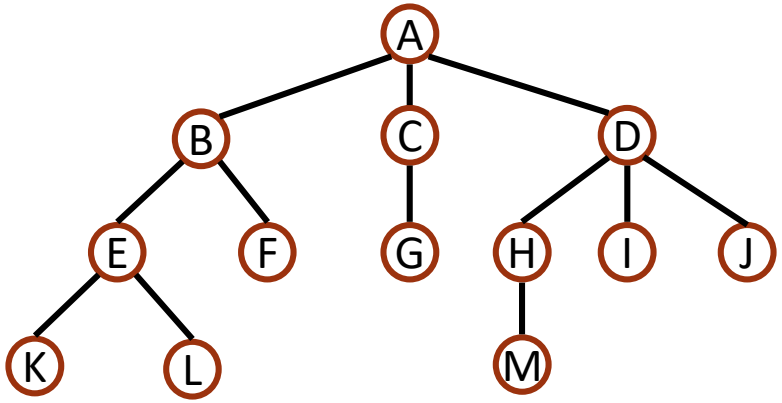
➢Using the left child-right sibling representation.

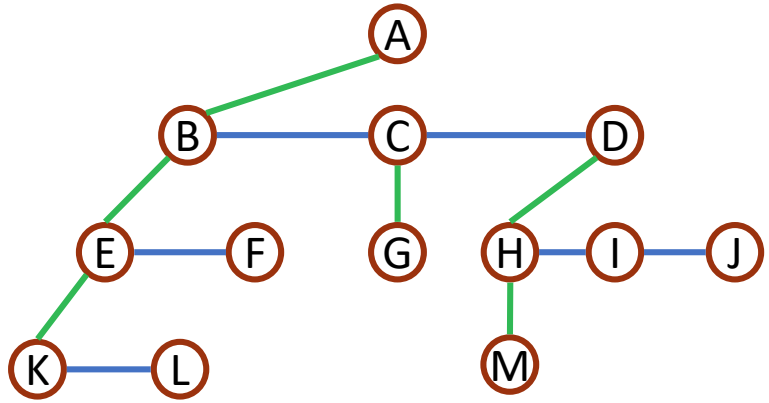


Representation of Trees – Left Child-Right Sibling Representation 2/2

- Every node has at most one leftmost child and at most one closest right sibling.
- The **left child** field of each node points to its leftmost child (if any), and the **right sibling** field points to its closest right sibling (if any).

data	
left child	right sibling



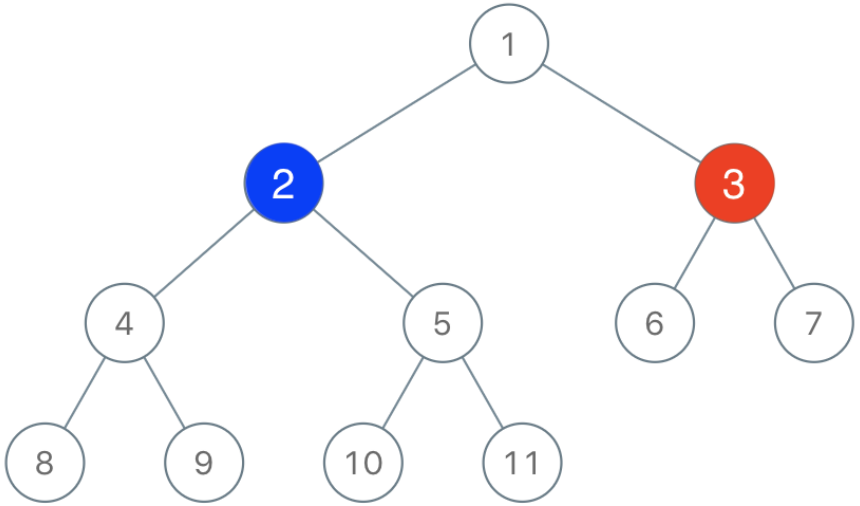


left child

right sibling

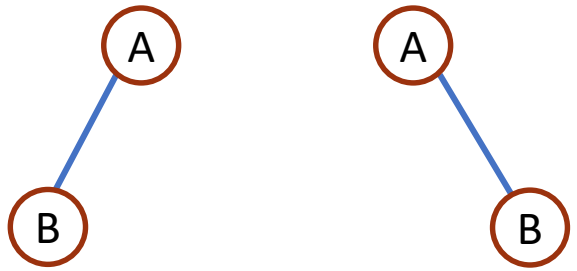
Binary Tree

- A **binary tree** is a finite set of nodes that is
  - ✓consists of **a root** and
  - ✓**two disjoint binary trees** called the left subtree and the right subtree.



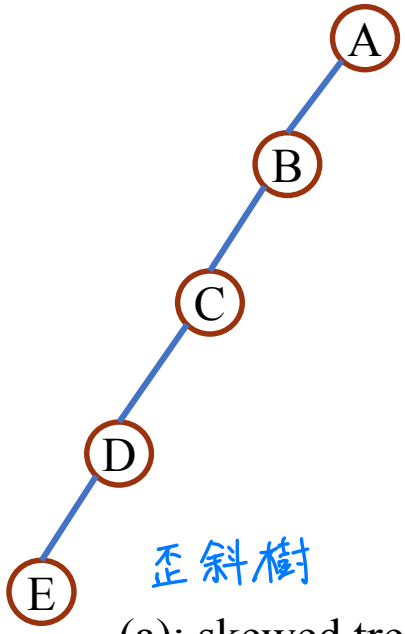
Binary Tree – tree v.s. binary tree

- The difference between a binary tree and a tree
  - ✓In a binary tree we distinguish between the order of the children while in a tree we do not.
  - ✓The following two binary trees are different since the first binary tree has an empty right subtree, while the second has an empty left subtree.

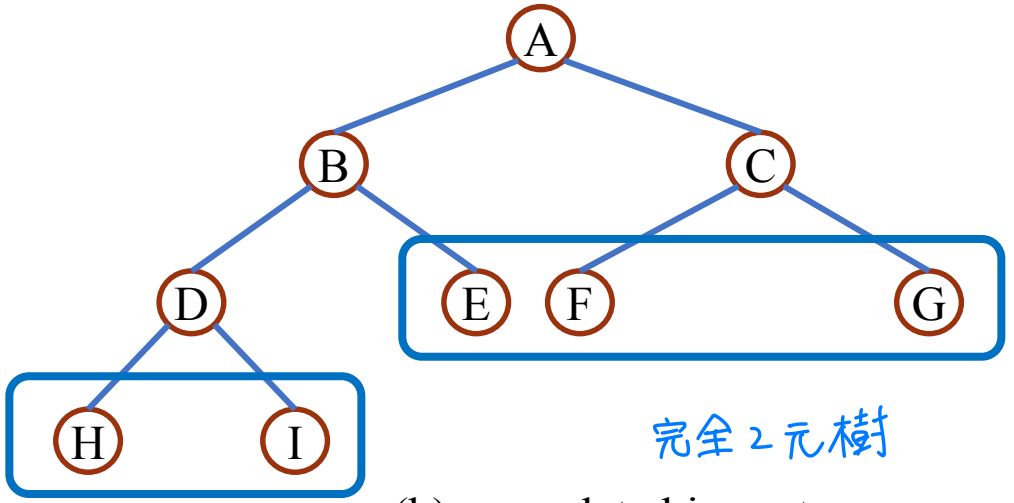


Binary Tree – skewed tree & complete binary tree

- Two special kinds of binary trees: **skewed tree** and **complete binary tree**.
- In a complete binary tree, all **leaf nodes** of these trees are on two adjacent levels.



(a): skewed tree



(b): complete binary tree

Properties of Binary Trees 1/2

➢ **Lemma 5.2** [**Maximum number of nodes**]:

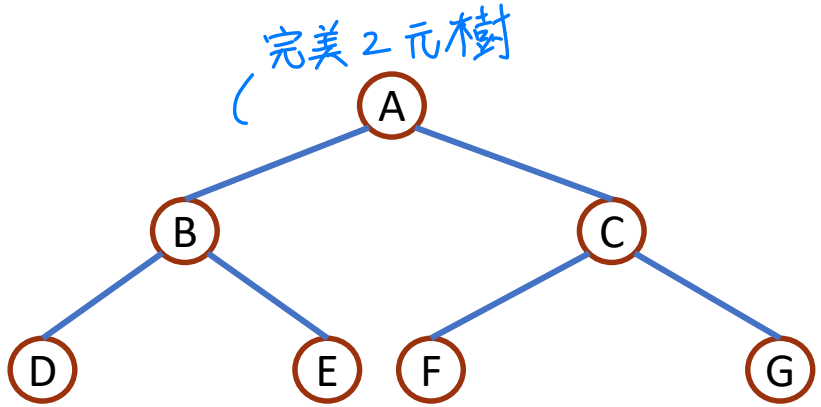
✓The maximum number of nodes on level i of a binary tree is 2^{i-1}, $i \geq 1$.

✓The maximum number of nodes in a binary tree of depth k is $\mathbf{2^k - 1}$, $k \geq 1$.

➢For example:

✓On level 1 = 1, level 2 = 2 and level 3 = 4.

✓The number of nodes of depth 3 is $2^3 - 1 = 7$.



Properties of Binary Trees 2/2

➢ **Proof** (1) : The Proof is by induction on i.

- ✓ Induction Base: The root is the only node on level 1. $2^{1-1} = 2^0 = 1$.
- ✓ Induction Hypothesis: Assume that the maximum number of nodes on level i-1 is 2^{i-2}.
- ✓ Induction Step:
  - The maximum number of nodes on level i-1 is 2^{i-2} by the induction hypothesis.
  - Since each node in a binary tree has a maximum degree of 2, the maximum number of nodes on level i is two times the maximum number of nodes on level i-1, that is $2^{i-2} * 2 = 2^{i-1}$.

➢ **Proof** (2) : The maximum number of nodes in a binary tree of depth k is

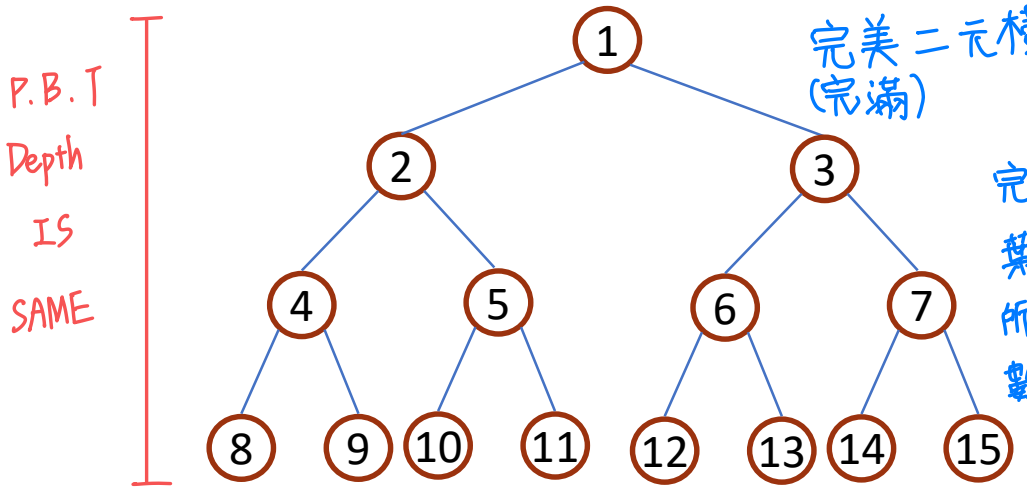
$$\sum_{i=1}^{k} (\text{maximum number of nodes on level } i) = \sum_{i=1}^{k} 2^{i-1} = \mathbf{2^k\text{-}1}.$$

$$1 + 2 + 2^2 + 2^3 + \cdots + 2^{k-1} = \frac{1-2^k}{1-2} = 2^k\text{-}1$$

Binary Tree – Full v.s. Complete

完全二元樹(C.P.T):
只有最下面2層結點的度數可以小於2,且最下面一層的結點都集中在該層左邊的連續位置上

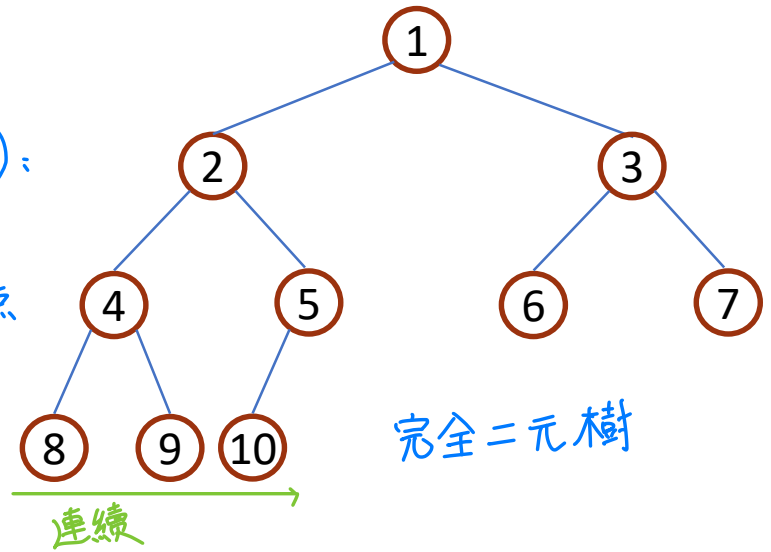
- A **full binary tree** of depth k is a binary tree of depth k having 2^k-1 nodes, $k \geq 0$.
- A binary tree with n nodes and depth k is **complete** iff its nodes correspond to the nodes numbered from 1 to n in the full binary tree of depth k.
  - ✓ From Lemma 5.2, it follows that the height of a complete binary tree with n nodes is $\lceil \log_2(n+1) \rceil$.



完美二元樹(完滿)

完美二元樹(P.B.T):
葉結點深度都一樣,
所有非葉結點的子結點數量均為2。

A **full** binary tree of depth 4.



完全二元樹

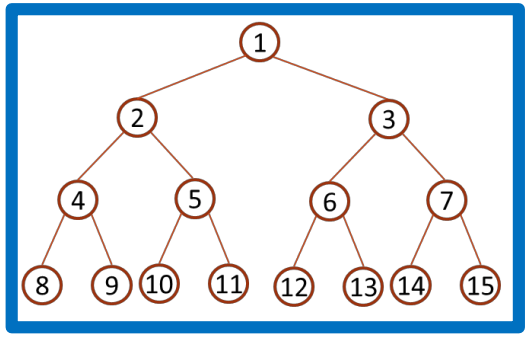
A **complete** binary tree with 10 nodes and depth 4.

Binary Tree Representations – Array 1/3

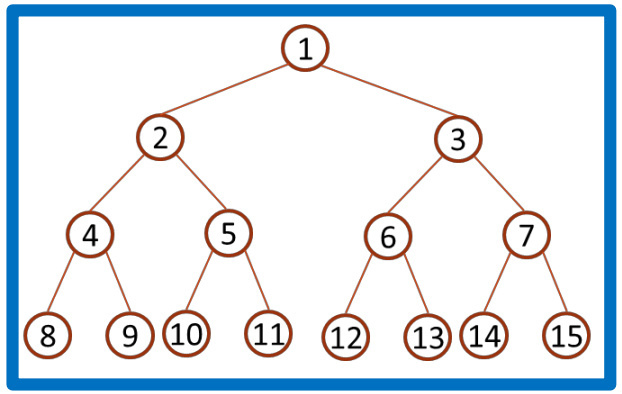
➢ Array Representation

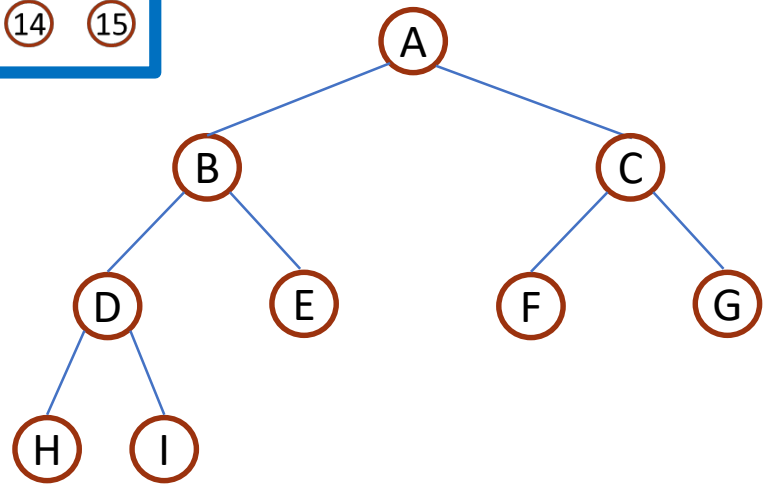
✓ **Lemma 5.4**: If a complete binary tree with n nodes is represented sequentially, then for any node with index i, $1 \le i \le n$, we have

- $parent(i)$ is at $\lfloor i/2 \rfloor$ if $i \neq 1$. If $i = 1$, i is at root and has no parent.
- $leftChild(i)$ is at $2i$ if $2i \le n$. If $2i > n$, then i has no left child.
- $rightChild(i)$ is at $2i+1$ if $2i+1 \le n$. If $2i+1 > n$, then i has no right child.



Binary Tree Representations – Array 2/3

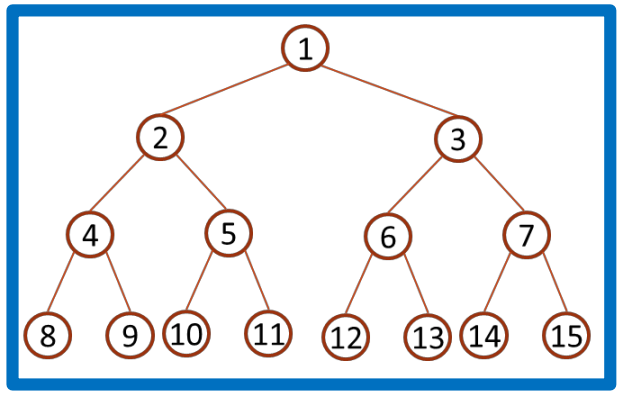


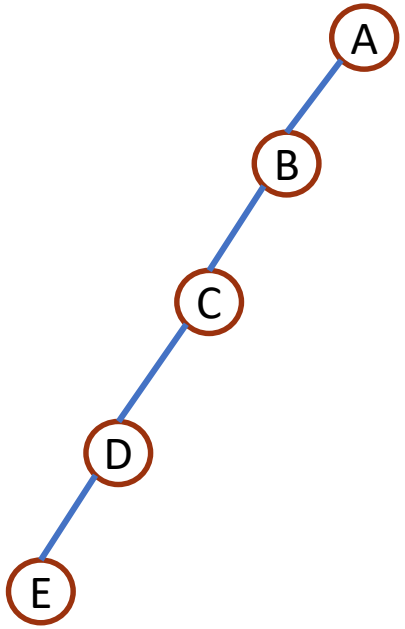


tree

[0]	
[1]	A
[2]	B
[3]	C
[4]	D
[5]	E
[6]	F
[7]	G
[8]	H
[9]	I

Binary Tree Representations – Array 3/3





tree

[0]	
[1]	A
[2]	B
[3]	
[4]	C
[5]	
[6]	
[7]	
[8]	D
[9]	
...	
[16]	E

Binary Tree Representations – Array's Drawbacks

- The drawbacks of array representation:
  - ✓**Waste memory** space for most binary trees.
  - ✓In the worst case, a skewed tree of depth k requires 2^k-1 spaces. Of these, only k spaces will be occupied.
  - ✓Insertion or deletion of nodes from the middle of a tree requires the **movement of potentially many nodes** to reflect the change in the level of these nodes

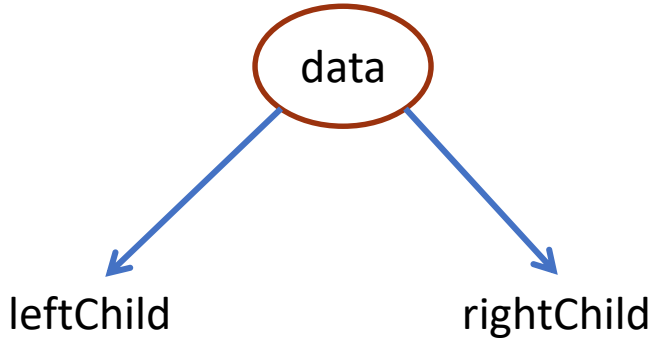
Binary Tree Representations – Linked List 1/3

➢**Linked list representation（鏈結串列表示法）**:

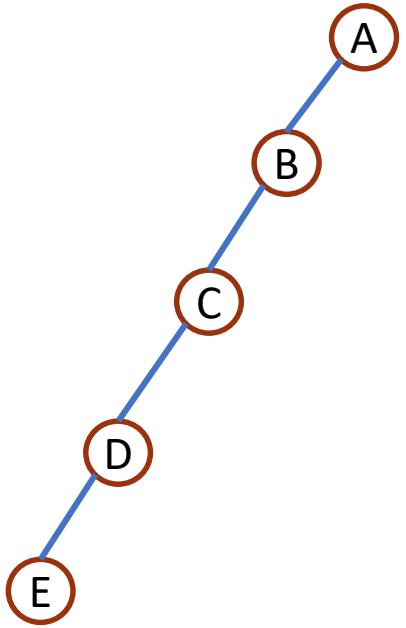
✓Each node has three fields: **leftChild, data**, and **rightChild**.

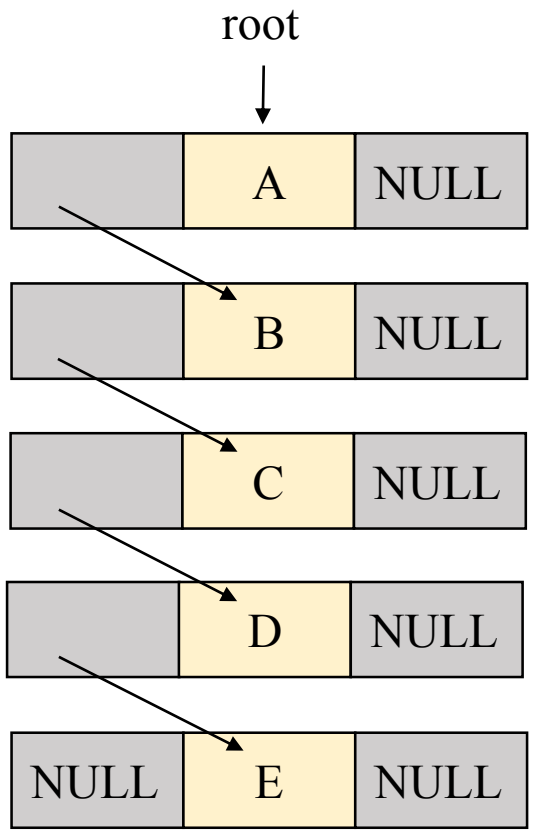
```
typedef struct node *treePointer;
typedef struct node {
        int data;
        treePointer leftChild, rightChild;
        };
```

leftChild	data	rightChild

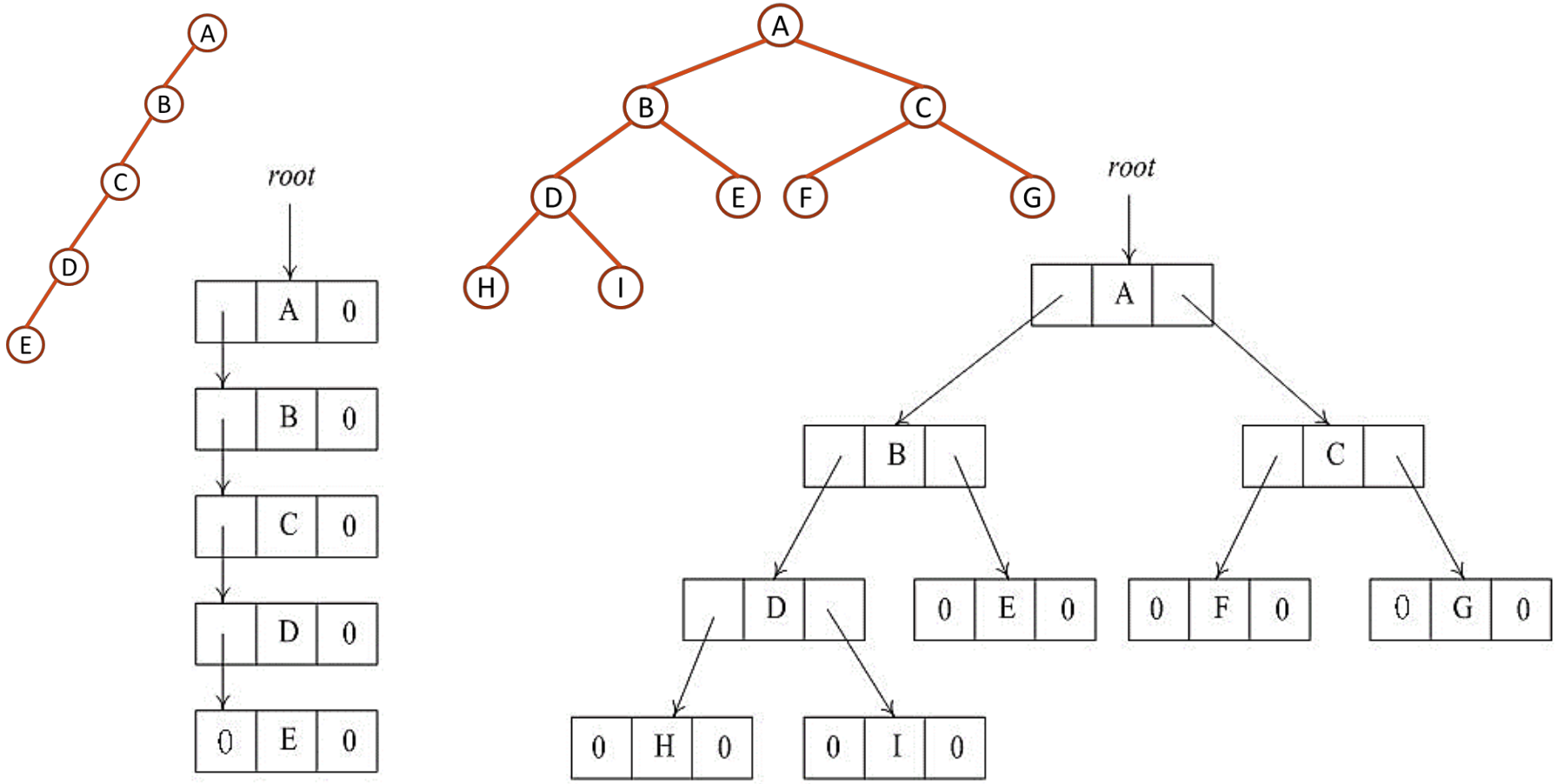


Binary Tree Representations – Linked List 2/3





Binary Tree Representations – Linked List 3/3



Binary Tree Traversals

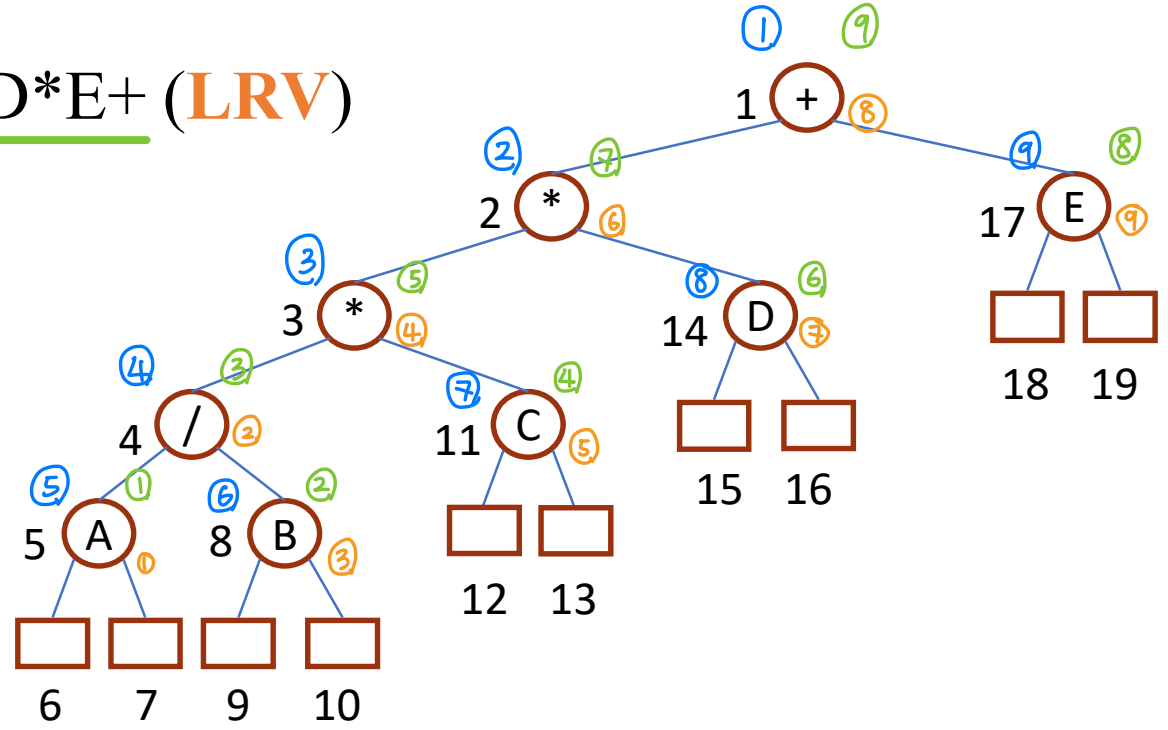
会考

- How to visit each node of a tree exactly once ?
- Let *L*, *V*, and *R* stand for moving left, visiting the node, and moving right when at a node.
  - ✓ There are six possible combinations of traversal.
    - **Eg., LVR, LRV, VLR, VRL, RVL, RLV**
- If we adopt the convention that we traverse left before right, then only three traversals remain:
  - ✓ **inorder**: *LVR* （中序走訪）
  - ✓ **postorder**: *LRV* （後序走訪）
  - ✓ **preorder**: *VLR* （前序走法）

Tree Traversal

会考

- Inorder traversal (中序走訪): A/B*C*D+E (**LVR**)
  - ✓Infix form
- Preorder traversal (前序走訪): +**/ABCDE (**VLR**)
  - ✓Prefix form
- Postorder traversal (後序走訪): AB/C*D*E+ (**LRV**)
  - ✓Postfix form



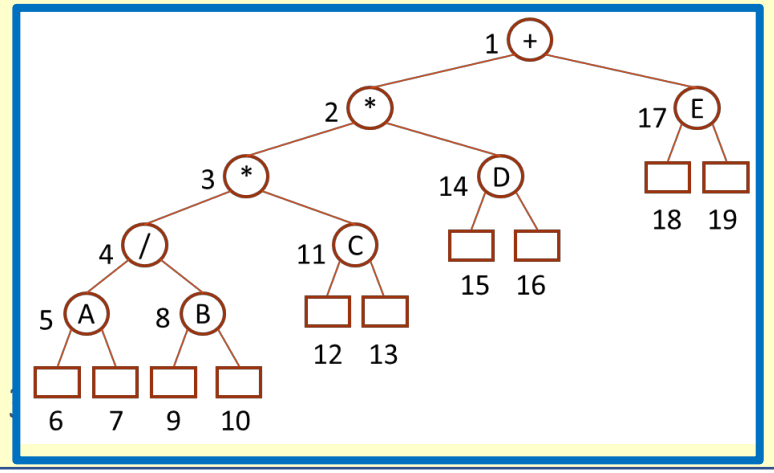
Program 5.1 Inorder traversal of a binary tree

会考

```
void inorder (treePointer ptr)
{
    /* inorder tree traversal */
    if (ptr) {
        inorder (ptr→leftChild);
        printf ("%d", ptr→data);
        inorder (ptr→rightChild);
    }
}
```

Call of inorder	Value in root	Action	Call of inorder	Value in root	Action
1	+		11	C	
2	*		12	NULL	
3	*		11	C	printf
4	/		13	NULL	
5	A		2	*	printf
6	NULL		14	D	
5	A	printf	15	NULL	
7	NULL		14	D	printf
4	/	printf	16	NULL	
8	B		1	+	printf
9	NULL		17	E	
8	B	printf	18	NULL	
10	NULL		17	E	printf
3	*	printf	19	NULL	

- Note that recursion is an elegant device for describing this traversal.



Inorder traversal: A/B*C*D+E (**LVR**)
Infix form

Preorder and Postorder traversals of a binary tree

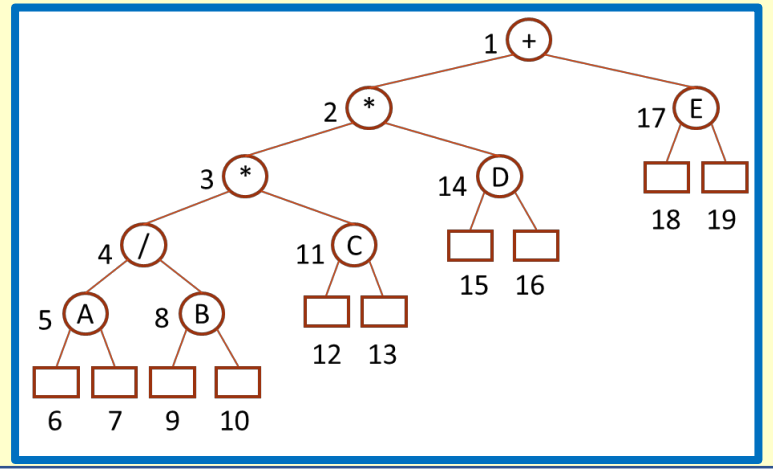
- void preorder (treePointer ptr)

```
{
   /* preorder tree traversal */
   if (ptr)  {
      printf ("%d", ptr→data);
      preorder (ptr→leftChild);
      preorder (ptr→rightChild);
    }
}
```

- void postorder (treePointer ptr)

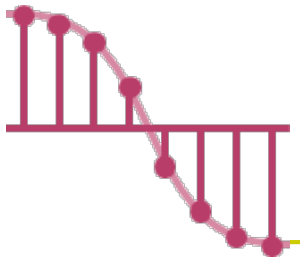
```
{
   /* postorder tree traversal */
   if (ptr)  {
     postorder(ptr→leftChild);
     postorder(ptr→rightChild);
     printf ("%d",ptr→data);
   }
}
```

以前考过



Preorder traversal: +**/ABCDE (**VLR**)
Prefix form

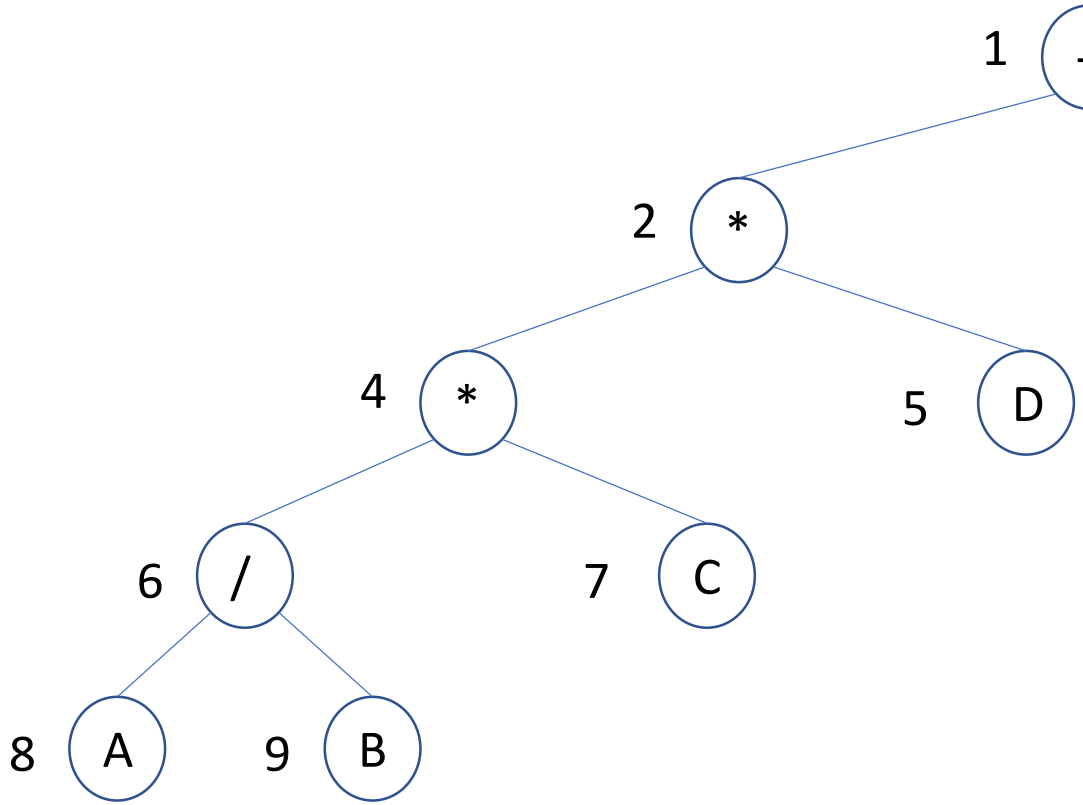
Postorder traversal: AB/C*D*E+ (**LRV**)
Postfix form



Level-Order Traversal

- When written recursively, the inorder, preorder, and postorder traversals all require a **stack**.
- We now turn to a traversal that requires a **queue**. This traversal called **level-order traversal**.
- The steps of level-order traversal are:
  - ✓visit the root first
  - ✓then the root's left child followed by the right child
  - ✓visit next level from leftmost node to right most node

An example for level-order Traversal

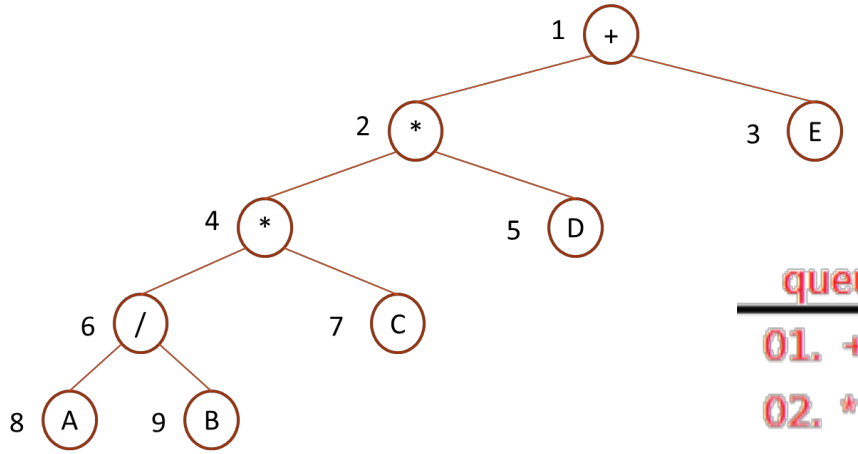


queue	printf
01. +	
02. *E	+
03. E*D	+*
04. *D	+*E
05. D/C	+*E*
06. /C	+*E*D
07. CAB	+*E*D/
08. AB	+*E*D/C
09. B	+*E*D/CA
10.	+*E*D/CAB

Level-order traversal: +*E*D/CAB

Program 5.5 Level-order traversal of a binary tree

```
void levelOrder(treePointer ptr)
{
    /* level order tree traversal */
    int front = rear = 0;
    treePointer queue[MAX_QUEUE_SIZE];
    if (!ptr) return; /* empty tree */
    addq(ptr);
    for (;;) {
        ptr = deleteq();
        if (ptr) {
            printf("%d",ptr→data);
            if (ptr→leftChild)
                addq(ptr→leftChild);
            if (ptr→rightChild)
                addq(ptr→rightChild);
        }
        else break;
    }
}
```



queue	printf
01. +	
02. *E	+
03. E*D	+*
04. *D	+*E
05. D/C	+*E*
06. /C	+*E*D
07. CAB	+*E*D/
08. AB	+*E*D/C
09. B	+*E*D/CA
10.	+*E*D/CAB

Discussion of Program 5.5

- The code assumes a **circular queue** as in Chapter 3.
- We begin by adding the root to the queue.
- The function operates by deleting the node at the front of the queue, printing out node's data field, and adding the node's left and right children to the queue.

Program 5.6 Copying a binary tree

- treePointer **copy(treePointer original)**

```
{
    /* this function returns a tree_pointer to an exact copy of the original tree */
    treePointer temp;
    if (original) {
        MALLOC(temp, sizeof(*temp));
        temp→leftChild = copy(original→leftChild);
        temp→rightChild = copy(original→rightChild);
        temp→data = original→data;
        return temp;
    }
    return NULL;
}
```

Program 5.7 Testing for equality of binary trees

- int **equal(treePointer first, treePointer second)**

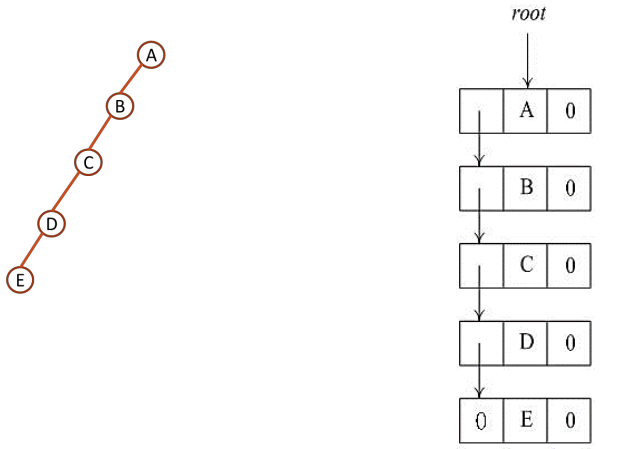
```
{
    /* function returns FALSE if the binary trees first and second are not equal,
    Otherwise it returns TRUE */
    return (
        (!first && !second) || (first && second &&
                (first→data == second→data) &&
                equal(first→leftChild, second→leftChild) &&
                equal(first→rightChild, second→rightChild)
    )
}
```

Threaded Binary Tree 1/2

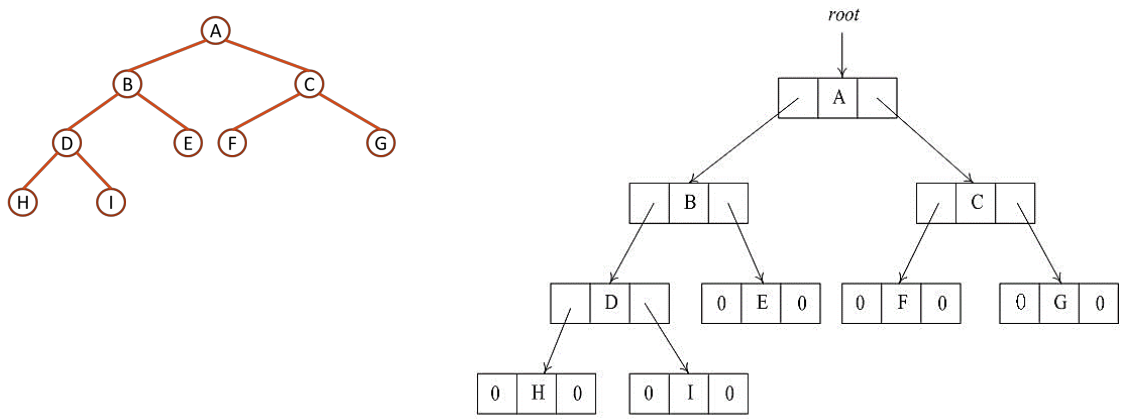
➢ Problem: there are more null links than actual points.

- ✓ Number of nodes: n
- ✓ Number of non-null links : n – 1
- ✓ Total links: 2n
- ✓ Therefore, the number of null links is 2n – (n–1) = n+1

Number of nodes: 5
Number of non-null links : 4
Number of null links: 6



Number of nodes: 9
Number of non-null links : 8
Number of null links: 10



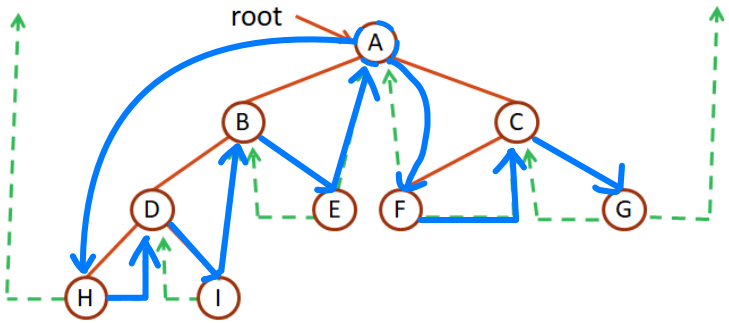
Threaded Binary Tree 2/2

- Problem: there are more null links than actual points.
  - ✓ Number of nodes: n
  - ✓ Number of non-null links : n – 1
  - ✓ Total links: 2n
  - ✓ Therefore, the number of null links is 2n – (n–1) = n+1

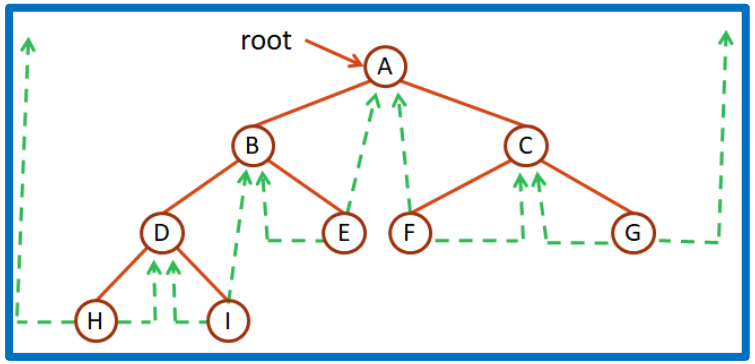
中序(LVR):

H→D→I→B→E→A→F→C→G

Solution: replace the null links by pointers, called **threads**.



Threaded Binary Tree - Rules

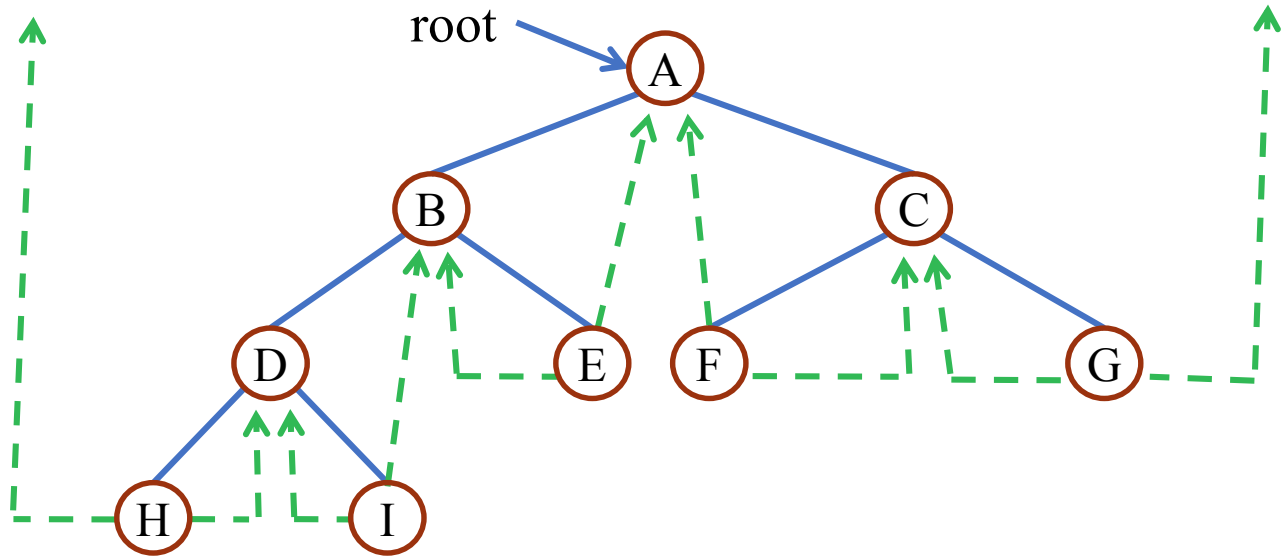


- Threading rules
  - ✓if ptr→leftChild is NULL, then

    ptr→leftChild = **inorder predecessor** of ptr.
    - That is, we replace the null link with a pointer to the inorder predecessor of ptr.
  - ✓if ptr→rightChild is NULL, then

    ptr→rightChild = **inorder successor** of ptr.
    - That is we replace the null link with a pointer to the inorder successor of ptr.

Figure 5.21 Threaded Tree



Inorder sequence: H D I B E A F C G

Threaded 1/3

- To distinguish between normal pointers and threads
  - ✓Two additional fields of the node structure, left-thread and right-thread

```
typedef struct threadedTree *threadedPointer;
typedef struct threadedTree {
        short int leftThread;
        threadedPointer leftChild;
        char data;
        threadedPointer rightChild;
        short int rightThread;
        };
```

Threaded 2/3

- If ptr→leftThread = TRUE, ptr → leftChild contains a thread;

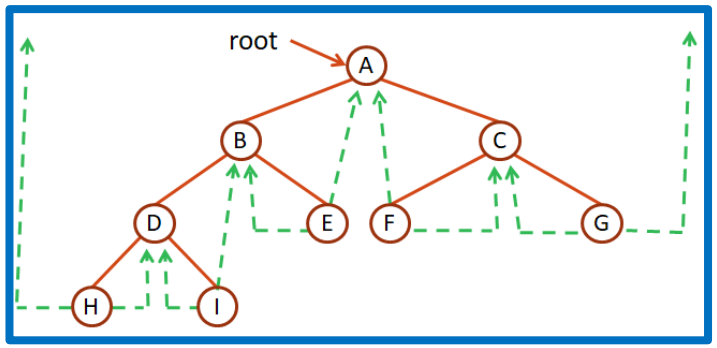
  Otherwise, it contains a pointer to the left child.

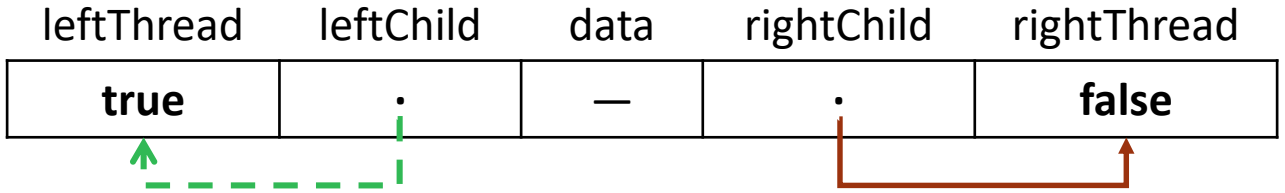
- Similarly, if ptr→rightThread = TRUE, ptr→ rightChild contains a thread;

  Otherwise, it contains a pointer to the right child.

Threaded 3/3

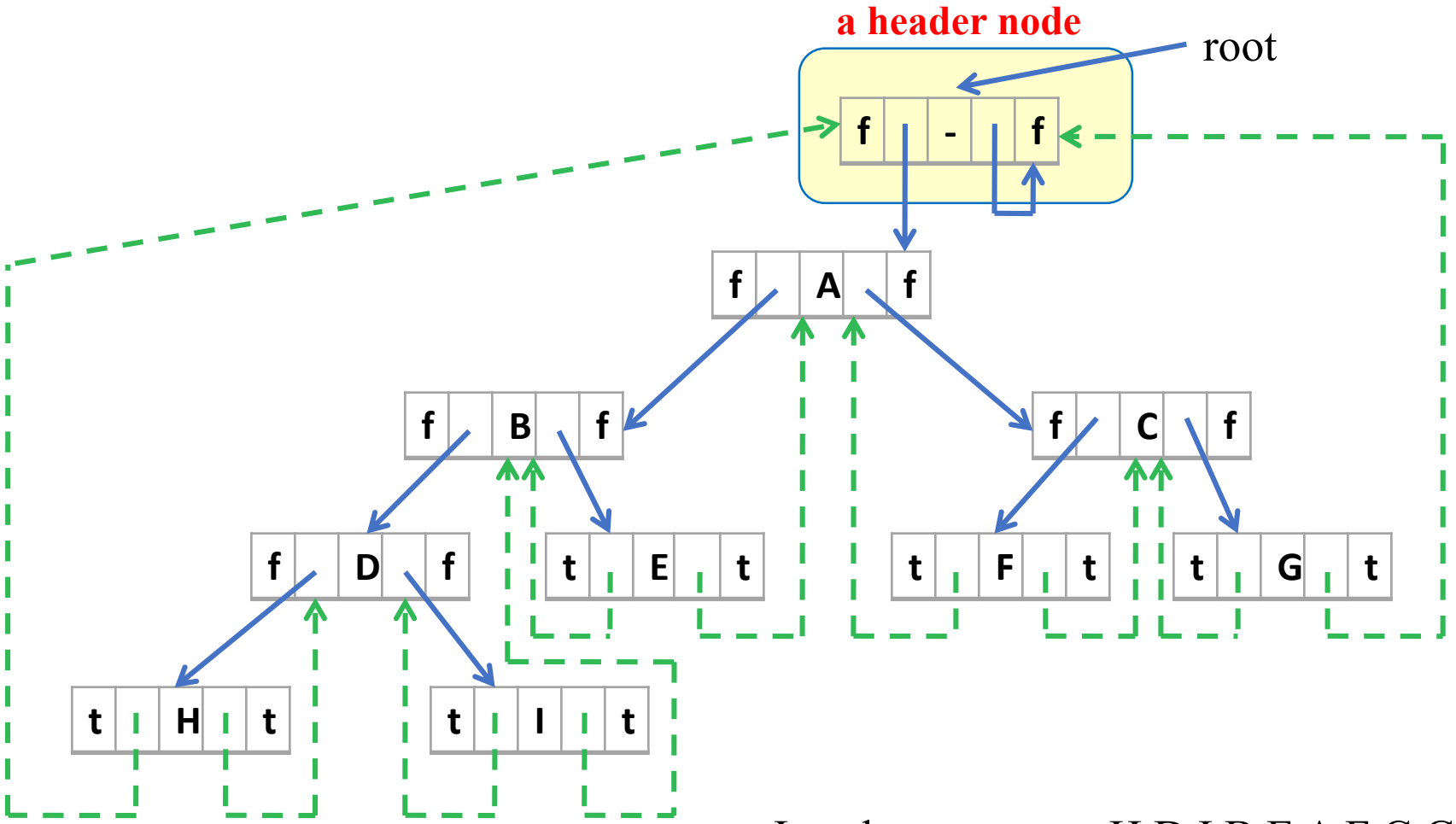
- In Fig 5.21 two threads have been left dangling: one in the left child of H, the other in the right child of G.
- To avoid dangling threads, we will assume a header node for all threaded binary tree.
- **The original tree becomes the left subtree of the head node.**
- An empty binary tree is represented by its header node as the following.





Memory representation of threaded tree of Figure 5.21

会出現在 Hw
可能会考！

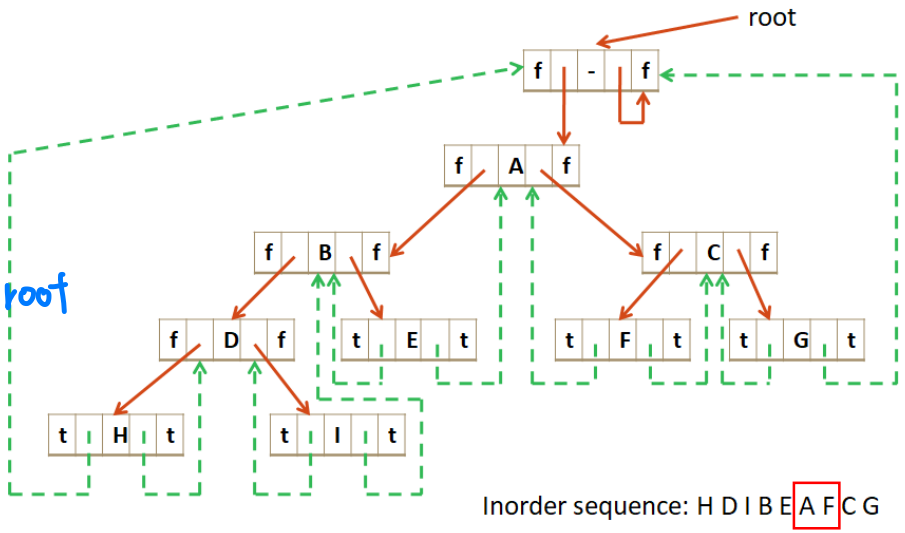


Inorder sequence: H D I B E A F C G

Program 5.10 Finding the Inorder Successor of a Node

➢ threadedPointer insucc(threadedPointer tree)

```
{
    /* find the inorder sucessor of tree in a threaded binary tree */
    threadedPointer temp;
    temp = tree→rightChild;
    if (!tree→rightThread)   // 表示有右子節點
        while (!temp→leftThread)  →走到沒有引線時，下一個就是root
            temp = temp→leftChild;
    return temp;
}
```

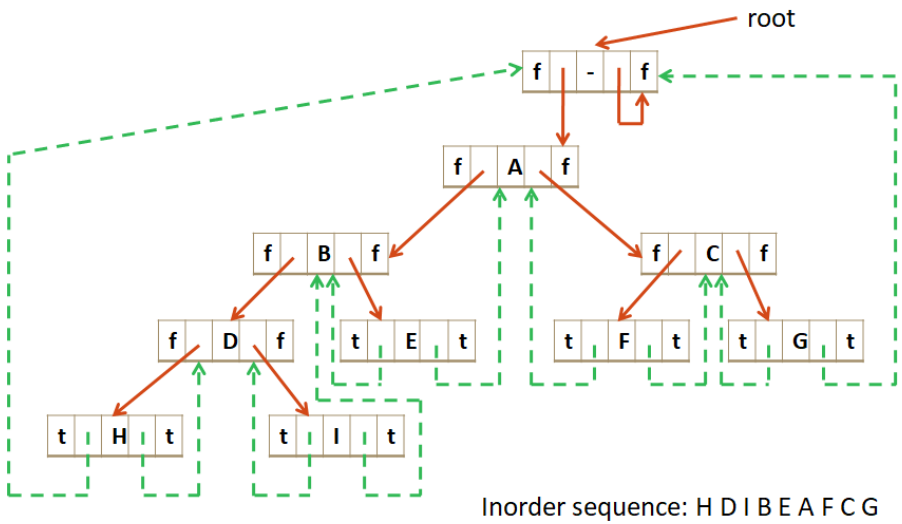


Inorder Traversal of a Threaded Binary Tree

- If ptr→rightThread = TRUE, the inorder successor of ptr is ptr→rightChild by definition of the threads
- Otherwise, we obtain the inorder successor of ptr by following a path of left-child links from the right-child of ptr until we reach a node with leftThread = TRUE.
- **To perform an inorder traversal we make repeated calls to insucc.**

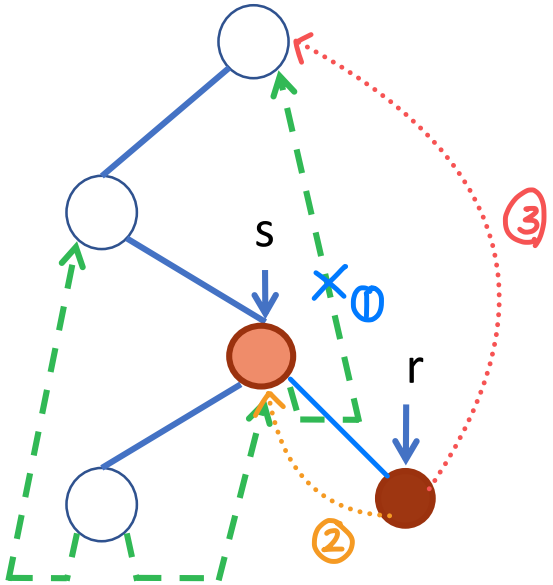
Program 5.11 Inorder traversal of a threaded binary tree

```
void tinorder(threadedPointer tree)
{
    /* traverse the threaded binary tree inorder */
    threadedPointer temp = tree;
    for (;;) {
        temp = insucc(temp);
        if (temp == tree)
                break;
        printf("%3c", temp->data);
    }
}
```

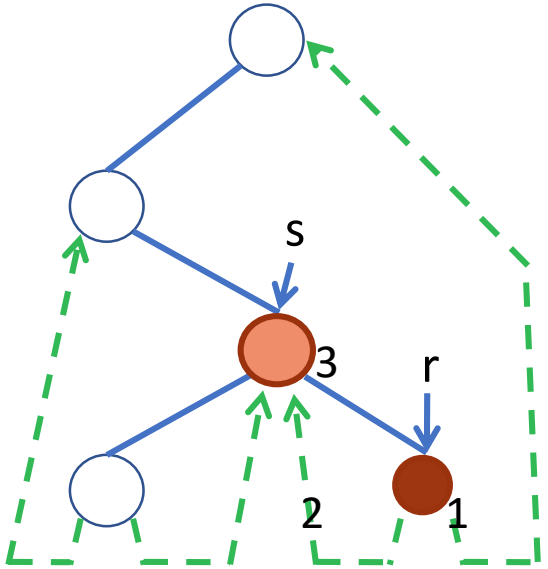


Inserting *r* as the right child a node *s*

- If *s* has an empty right subtree, then the insertion is simple and diagrammed in Fig 5.24(a).



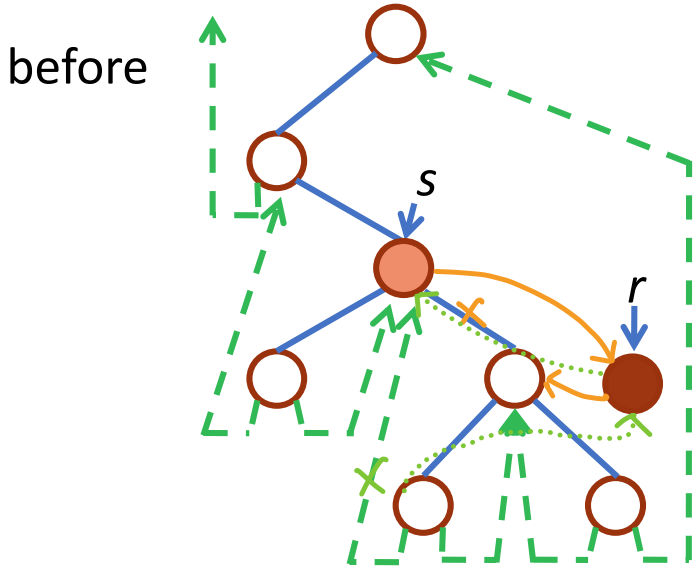
before

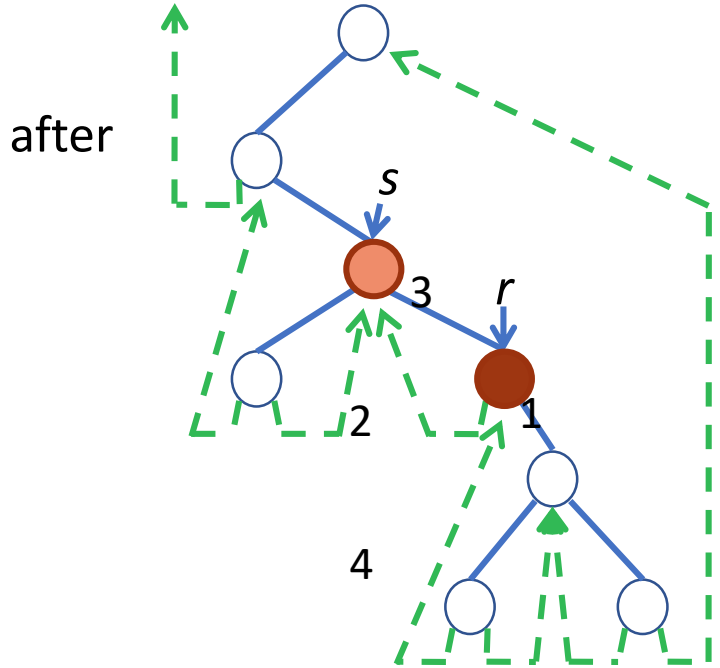


after

Inserting *r* as the right child a node *s*

- If the right subtree of s is not empty, then
  - ✓ the right subtree of *s* is made the right subtree of *r*
  - ✓ *r* becomes the inorder predecessor of a node which was previously the inorder successor of *s*
  - ✓ the insertion is diagrammed in Fig 5.24(b)





Program 5.12 Right Insertion in a threaded binary tree

- void insertRight (threadedPointer s, threadedPointer r)

```
{
    /* insert r as the right child of s */
    threadedPointer temp;
    r→rightChild = s→rightChild;
    r→rightThread = s→rightThread;
    r→leftChild = s;
    r→leftThread = TRUE;
    s→rightChild = r;
    s→rightThread = FALSE;
    if (!r→rightThread){      // 4
            temp = insucc(r);
            temp→leftChild=r;
    }
}
```